Re-Profiling `mandelHybrid` After the `examine()` Min/Max Bug Fix

Matias Saavedra

Friday 10th July, 2026

Binary: d5bf30c (tag binary-v1-bugfix) — after the one-line fix to MandelRegion::examine()'s min/max reduction.

Resumen

This report re-runs the Nsight Systems profile of `mandelHybrid` after the one-line bug fix to `MandelRegion::examine()`'s min/max tracking loop documented in `03-bug-analysis.tex`, and compares the new measurements against the pre-fix baseline of `01-initial-profile.tex`. The methodology is identical: 100 frames at 1920×1080 on an i7-9750H + GTX 1660 Ti Max-Q with 12 threads (1 GPU + 11 CPU), invoked as `nsys profile --trace=cuda,nvtx,osrt -o report ./mandelHybrid spec.in`. The fix produced visible, mostly-expected changes in the work-decomposition metrics: +12% more `examine()` calls (6,104 → 6,836), +12% more leaf regions (4,603 → 5,152, with the CPU pool absorbing most of the increase: 1,060 → 1,414), -26% average CPU region time (535 ms → 398 ms), and -70% in the GPU region maximum (465.7 ms → 139.4 ms). The single most striking finding, however, is that the 15.3 s CPU outlier survived the fix, landing at 14.2 s in the new run. The bug fix is therefore a correctness improvement that does what its analysis predicted (more splits, tighter tails on the GPU side), but the long-tail CPU outlier is now confirmed to be caused by the intrinsic weakness of the four-corner heuristic rather than the tracking bug.

1. Methodology

Identical to the pre-fix run that produced `experiments/01-initial-profile/report.nsys-rep`: the same `spec.in` (100 frames at 1920×1080, the same zoom trajectory ending at `maxIterations` = 10,000), the same machine (i7-9750H, 6 physical / 12 logical cores; GTX 1660 Ti Max-Q; nvcc 13.2 driving g++-15 for the host compile), and the same defaults for `numThreads` (12, the machine's logical core count), `gpuEnable` (1), `diffThreshold` (0.5), and `pixelSizeThresh` (32,768). The only differences are:

- the binary now contains the one-line fix to `MandelRegion::examine()`'s reduction loop:

Código 1: mandelregion.cpp — the fix being measured.

```
// Track min and max independently. The original code chained these
// with `else if`, which made max-tracking unreachable whenever the
// current sample also lowered the min -- most notably on i==0, where
// minIter starts at INT_MAX so the min branch always fires and
```

```
// maxIter is never updated from the first corner.
if (cornersIter[i] < minIter)
  minIter = cornersIter[i];
if (cornersIter[i] > maxIter)
  maxIter = cornersIter[i];
```

- the new `report.nsys-rep` lives in `experiments/04-postfix-profile/` so the pre-fix report in `experiments/01-initial-profile/` remains untouched as a baseline.

The invocation was `nsys profile --trace=cuda,nvtx,osrt -o report ../mandelHybrid ../spec.in`, run from `experiments/04-postfix-profile/` so the `img`$NNNN$`.png` outputs and the profile report land together. PNGs are cleaned afterward; the `report.nsys-rep` and the `report.sqlite` export it generates are kept.

2. Results

2.1. NVTX Range Summary

Table 1 is the direct analogue of Table 1 of the pre-fix report. All times are summed across all threads and all instances; the `Range` column lists the three NVTX labels added to the program in the original instrumentation pass.

Tabla 1: NVTX range summary, post-fix run (summed across all threads).

Range	Time%	Total (s)	Instances	Avg (ms)	Med (ms)	Max (ms)
`examine region`	50.9%	633.05	6,836	92.61	2.29	14,214
`CPU region compute`	45.3%	563.22	1,414	398.32	56.29	14,214
`GPU region compute`	3.8%	47.79	3,738	12.78	4.80	139.4

2.2. Per-Thread CPU Work Distribution

Per-thread totals were extracted from the `[CPU profiling summary - this thread]` lines printed when each worker drains the queue. Threads are listed in the order they finished (the order is not stable run-to-run; numbering is for reference only).

Tabla 2: CPU region distribution across the 11 CPU threads, post-fix.

Thread	Regions	Total (s)	Avg (ms/region)
1	153	51.17	334.5
2	110	51.71	470.1
3	91	50.67	556.8
4	179	51.22	286.2
5	115	51.39	446.9
6	93	51.17	550.2
7	175	51.69	295.4
8	138	50.80	368.1
9	130	50.19	386.1
10	120	50.94	424.5
11	110	52.19	474.5
Total CPU	1,414	—	398.3
GPU thread	3,738	47.79	12.8
End-to-end wall	—	53.24	—

2.3. CUDA API and GPU Kernel Summary

Tabla 3: CUDA API summary, post-fix (host-side, driver perspective).

API call	Time%	Calls	Avg	Total
`cudaMemcpy`	99.2%	3,738	12.7 ms	47.4 s
`cudaLaunchKernel`	0.1%	3,738	15.3 µs	57.1 ms
`cudaEventRecord`	0.1%	14,952	4.5 µs	66.9 ms
`cudaEventSynchronize`	0.0%	3,738	4.8 µs	17.9 ms

Tabla 4: GPU kernel and D→H memcpy times, post-fix (CUPTI hardware counters).

Operation	Count	Total (s)	Avg	Med	Min	Max
`mandelKernel`	3,738	46.50	12.44 ms	4.55 ms	8.2 µs	138.9 ms
D→H memcpy	3,738	0.090	24.0 µs	20.0 µs	19.5 µs	90.4 µs

3. Pre-Fix vs. Post-Fix Comparison

Table 5 puts the headline numbers side by side.

Tabla 5: Pre-fix vs. post-fix headline metrics on the same machine, same spec, same default thresholds.

Metric	Pre-fix	Post-fix	Δ
`examine()` calls	6,104	6,836	+12.0%
Total leaf regions	4,603	5,152	+11.9%
of which CPU-computed	1,060	1,414	+33.4%
of which GPU-computed	3,543	3,738	+5.5%
Avg CPU compute / region (ms)	535	398	-25.6%
Avg GPU compute / region (ms)	13.5	12.8	-5.2%
Max CPU region (s)	15.3	14.2	-7.1%
Max GPU region (ms)	465.7	139.4	-70.1%
CPU thread wall spread (s)	50.50–52.59	50.19–52.19	similar
spread %	4.1%	3.8%	similar
GPU thread wall (s)	47.11	47.79	+1.4%
End-to-end elapsed (s)	∼52	53.24	similar
`examine()` overhead/call (ms)	3.95	3.22	-18.5%
D→H transfer avg (µs)	15.0	24.0	+60.0%

3.1. What Changed and Why

More splits, smaller pieces.

The fix exposed regions that the buggy reduction had silently classified as "uniform" when in fact a high corner value (or a value not in iteration order) had been discarded. Each such misclassification now correctly triggers a four-way split, producing additional leaves. The +12% in `examine()` calls and the +12% in leaf count are consistent with the back-of-envelope estimate from `03-bug-analysis.tex` that ∼25% of regions had their maximum dropped, of which a fraction crossed the `diffThresh` threshold.

CPU pool absorbs most of the new splits.

GPU leaves grew by +5.5% but CPU leaves grew by +33.4%. This is exactly what the work-queue scheduler predicts: when the average region shrinks, the GPU's per-region wins shrink in absolute terms (small regions launch a kernel on a small grid; the kernel finishes quickly and the GPU has to come back to the queue to pull another item, leaving more small items for the CPU workers). The CPU's 40× slower per-region rate (Section 4.1 of the pre-fix report) means the CPU naturally claims any work that is small enough for the GPU's per-call overhead to dominate.

Average per-region time fell on both sides.

The CPU average dropped 26% (535 → 398 ms) and the GPU average dropped 5% (13.5 → 12.8 ms). The asymmetry is again a small-region effect: the new splits subdivide regions that were previously computed as one block, so the per-region figure now includes more medium-sized pieces and fewer mid-large pieces.

GPU long tail nearly disappeared; CPU long tail did not.

The GPU's worst-case region fell by 70%, from 465.7 ms down to 139.4 ms. This was exactly the kind of region the bug was hiding: a high-iteration region whose

maximum-corner happened to be discarded by the buggy reduction, dispatched as a single block to the GPU. Splitting it into four (and recursing) caps the GPU's per-launch tail. The CPU's worst-case region, on the other hand, only fell from 15.3 s to 14.2 s (-7%). This residual outlier is the same pattern that `03-bug-analysis.tex` called out as not explainable by the bug: a region whose interior contains a high-iteration filament that genuinely does not touch any of the four corners. No reduction fix can help here; the heuristic itself is the limiting factor, and the follow-ups suggested in the original profiling report (lower `diffThreshold`, edge-midpoint sampling, cardioid certificates) are now the only remaining levers.

Load balance: marginally tighter, but unchanged in character.

The 11-thread spread improved from 50.50–52.59 s (4.1%) to 50.19–52.19 s (3.8%) — a small improvement. Most of the per-thread variance comes from the same outlier region as before, just now slightly smaller and absorbed by a different thread. The end-to-end wall time is essentially unchanged (53.24 s post-fix; the pre-fix headline figure was "∼52 s" inferred from the per-thread totals).

`examine()` per-call overhead actually dropped.

The table shows pure subdivision overhead (`examine` − CPU − GPU) falling from 3.95 ms/call to 3.22 ms/call (-18%). This is at first counter-intuitive — if the fix exposes more splits, you might expect more corner-evaluation work. The opposite is the case because the bulk of the new splits happen deeper in the quad-tree: each child inherits one of its parent's corners (see lines 228–237 of `mandelregion.cpp`), so a deep split spends one `diverge()` call per new corner needed, not four. The new splits therefore add cheap leaves to the existing tree, not new roots; per-call overhead reflects an average tilted slightly toward already-cached corners.

D→H transfer time went up.

The CUPTI memcpy counter average rose from 15 µs to 24 µs (+60%). This is not a regression; it is a small-region effect. PCIe transfers have a fixed launch latency (driver setup, ring-buffer enqueue, DMA descriptor preparation) on top of the per-byte cost. As regions get smaller, the fixed cost amortizes over fewer bytes and the per-call average rises. The aggregate transfer time (89.6 ms versus the pre-fix 53 ms) is still negligible against the kernel time (46.5 s); the conclusion of the pre-fix report — "`cudaMemcpy` spends 99.9% of its reported time waiting for the kernel, not transferring bytes" — still holds.

3.2. Worst-Case Region: Bug or Heuristic?

The pre-fix report attributed the 15.3 s CPU outlier to the corner-heuristic weakness; `03-bug-analysis.tex` added the alternative explanation that the reduction bug may have been either the sole cause or a contributing factor. With the post-fix measurement now in hand, the question is resolved: the outlier is predominantly caused by the heuristic, with the bug contributing a modest reduction in worst-case duration. Quantitatively, the bug fix removed roughly $15.3 - 14.2 = 1.1$ s (7%) from the worst single region, which is consistent with the fix simply tightening the spread on a region whose corners were already misclassifying badly and where the discarded maximum was an incremental signal. The remaining 14.2 s is a region whose corner values are genuinely close to one another while its interior contains a long-iteration filament; the fractal boundary of the Mandelbrot set guarantees such regions exist at every scale.

4. Implications and Next Steps

The fix is a clean correctness improvement that delivers most of the predicted effects:

- It makes the four-corner heuristic do what its decision rule claims to do: split when the iteration spread is large. Roughly ∼12% of regions that were previously being computed in a single pass are now correctly being subdivided.
- The GPU tail dropped by 70%, eliminating the worst-case kernel launches that previously stalled thread 0 in `cudaMemcpy` for up to 465 ms.
- The CPU average dropped by 26% per region, lowering the blast radius of any single unlucky region claim from a queue extraction.
- Per-thread load balance tightened slightly (4.1% → 3.8% spread).

It does not change the end-to-end wall time at this thread configuration. The reason is that the bottleneck on this machine is neither queue throughput nor classifier accuracy — it is the single-threaded `maxIterations` = 10,000 inner loop hitting boundary points at the deepest zoom levels. Until the heuristic itself is improved, no amount of correctness work on the reduction can move the worst-case region below the seconds-scale.

4.1. Recommended Follow-Ups

The post-fix measurements sharpen the priority list of the improvements suggested in the pre-fix report:

1. Lower `diffThreshold`. With the bug fixed and the long-tail still seconds-scale, lowering `diffThreshold` from 0.5 to 0.2–0.3 is now the most direct way to attack the remaining 14 s outlier. The trade-off is more queue traffic; given that the per-call `examine()` overhead is already low (3 ms/call), the cost should be modest.
2. Edge-midpoint and 9-point stencil sampling. Sampling the edge midpoints (and possibly the centre) and inheriting those samples as the children's corners would catch interior filaments that miss all four corners — precisely the failure mode behind the 14.2 s leftover.
3. Cardioid/period-2 bulb interior certificate. For regions whose corners all satisfy the cardioid or main-bulb membership test, a constant-fill is free and exact. Most-zoom frames spend a significant fraction of their pixels inside the main cardioid; certifying them would skip thousands of unnecessary iterations per leaf.
4. Async CUDA streams. The pre-fix report's headline finding (`cudaMemcpy` blocks on kernel completion for 13 ms/region) is unchanged — still the largest unexplored optimisation. Async streams could overlap kernel n with copy of kernel $n-1$ and recover most of that 13 ms.

5. Conclusion

The one-line fix to `MandelRegion::examine()`'s min/max reduction is now confirmed by direct measurement to deliver the predicted effects: more splits (+12%), smaller pieces (CPU avg −26%), and a much tighter GPU tail (−70% on the maximum region). The end-to-end wall time is unchanged at this thread configuration because the binding constraint is the heuristic itself, not the reduction — a fact that the survival of the ∼14 s CPU outlier makes unambiguous. The previously-attributed cause of that outlier ("the corner heuristic misclassified a boundary region") is now correctly understood as a fractal-filament effect that no fix at the reduction layer can address. Future profiling effort should now focus on the four follow-ups listed in Section 4.1.